



IA en ingeniería con un esquema determinista de tres capas

➤ Por qué el 84 % de un sistema con LLM no debería contener un LLM

👤 ELECTRONICA 📅 2026-10-05 📁 51 directivas ⚙️ 62 scripts de ejecución 📡 23 flujos 📄 68 sesiones documentadas

1 1. La asimetría

Un modelo de lenguaje es un generador de texto plausible. La ingeniería no pide plausibilidad: pide que el mismo input produzca el mismo output y que se pueda auditar por qué. Esa asimetría no es una discusión filosófica, es medible, y se mide en fallos reales con coste concreto.

2 2. Las tres capas

Capa	Artefacto	Responde
1 · Directiva	directives/*.yaml	¿qué?
2 · Orquestación	flujo_*.py, mcp_*.py	¿cuándo?
3 · Ejecución	execution/*.py	¿cómo?

💡 Concepto clave

La regla. La lógica de negocio no se delega al modelo. Se le pide lo único que hace bien — producir texto — y el resto se calcula en código.

Capa 1, la directiva. Es configuración, no conversación: se versiona, se revisa con un diff y la escribe quien entiende el dominio.

```
1 # 62 scripts de ejecución
2 # 52 sin ninguna llamada a LLM <- 84 por ciento
3 # 10 con LLM (fronteras declaradas, archivos nombrados)
4 # el enrutador decide el tier localmente: 0 llamadas a la API, coste 0
```

Listing 1: La frontera LLM, medida sobre el disco

Recuerda

Criterio explícito, para que el número sea auditable. 13 scripts referencian maquinaria LLM; se excluyen ['enrutador.py', 'generar_charlada_latex.py', 'llm_client.py'] (cliente y enrutador: no invocan modelo).

3 3. Tres fallos que costaron dinero

3.1 El modelo puntuaba; el programa suma

El sistema devolvía `status: ok` y un JSON bien formado. Cada ítem llevaba su denominador correcto. El total *no era la suma de sus propias partes*: el modelo había inventado el campo y la aritmética era suya.

La causa no era de formato sino de **ROL**: se le pidió al LLM el total. Cambiar la prosa no lo arregla —se probó—: los denominadores pasaron a ser correctos mientras el total seguía sin cuadrar.

Ejemplo

El arreglo fue dejarle *una sola* tarea — puntuar ítems — y que el programa sumara. El máximo pasó a ser un invariante: si la suma se pasa del tope, se recorta y *se avisa*. Cada parcial se limita a su propio peso. Y lo ilegible se cuenta y se dice: un ítem sin parcial genera aviso, porque ignorarlo en silencio hace creer que el alumno contestó lo que no contestó.

3.2 La escala la declara el instrumento

Tres preguntas de un punto cada una (escala 3) contestadas a medias. Sumar en crudo daba **1.5/10** y nivel *Insuficiente*: el alumno había sacado el 50 %.

La escala de un examen no es 10: es la que declara el examen. La nota institucional sí es sobre 10, así que hace falta una conversión explícita.

Hubo además un **bug espejo**: al armar el denominador con lo que devolvía el modelo, un criterio que el modelo se comía desaparecía del denominador y todo lo demás se normalizaba *hacia arriba*. El alumno cobraba por un fallo del modelo. Con rúbrica, la escala la fija el archivo de configuración y un criterio ausente vale 0 sobre esa escala.

Concepto clave

Ausencia de dato no es dato. Sin escala declarada no se publica nota: se publica el motivo y la suma observada, que es información útil para el humano.

3.3 El modo JSON no es un esquema

`response_format={"type": "json_object"}` promete **una sola cosa**: que la respuesta sea un objeto. No promete las claves, ni los tipos, ni el vocabulario. El esquema del prompt es texto, y el texto es la capa más débil del contrato.

El agravante fue más tonto: la instrucción *nunca se envió*. El script construía el mensaje de usuario y se olvidaba de adjuntar el esquema, así que el modelo recibía literalmente *devuelve SOLO el JSON* — sin saber de qué JSON — y respondió con la primera forma que se le ocurrió. Y como `json.loads` no tiene nada que decir sobre claves faltantes, la inspección *pareció* válida.

Ejemplo

El arreglo no fue en el prompt: fue un **validador en el programa**, con vocabulario cerrado para los campos enumerados y comprobación de tipos para el resto. Y la clave que decide si el flujo es viable se volvió obligatoria: confundir *no saber* con *no contestar* convierte una inspección que no inspeccionó nada en un informe aparentemente confiable.

4 4. Una idea formal que se transfiere

La **álgebra de veredictos** compartida por las dos auditorías del proyecto: **gana el peor estado, nunca un promedio**, lo no verificado impide el verde, y se distinguen tres estados que la intuición colapsa en uno: *no instalado, caído y parado*.

El motivo por el que esto es una idea y no una función: un promedio con tres estados en rojo y uno en amarillo se ve verde mientras algo crítico no se midió. Y la segunda mitad de la regla es la que se olvida: una sola definición compartida. Dos álgebras que divergen en silencio son la forma más cara de equivocarse en un módulo cuya única razón de ser es la consistencia.

5 5. Los límites

La capa 1 todavía no es verificable por la auditoría: es informativa. Un motor de asistente rotativo no es reproducible. El hardware disponible no entrena modelos. Un patrón cuya frontera se declara se enseña mejor que uno que se vende.

6 6. Seis pasos para adoptarlo

1. Escriba el SOP antes del código. Si no sabe escribirlo, aún no sabe qué tiene que automatizar.

2. Ponga el criterio en el programa: toda aritmética, comparación o conversión vive en código, no en el prompt.
3. Valide la forma, no confíe en ella.
4. Defina la escala en el instrumento, y si no la hay no publique el número.
5. Verifique por registro, no por bandera verde: un programa que termina con código 0 puede estar mintiendo, así que hay que leer su salida.
6. Deje la frontera escrita: los ficheros con LLM se nombran.

Este material lo genera el propio patrón del que habla: directiva, orquestador y script, con la paleta de color compartida del workspace.